has to be sent over the wire. Instead, you want to ask the database to run queries like this for you.

The query manager provides the `.filter()` method for accessing a subset of the objects from a table. The arguments to `.filter()` are key–value pairs, where the keys are the fields you are searching and the values are the content you are looking for. To find all the musicians with the first name *John*, you call `.filter()` using `first_name="John"` as the argument:

```
>>> Musician.objects.filter(first_name="John")
<QuerySet [<Musician: Musician(id=2, last_name=Lennon)>, <Musician:
➥ Musician(id=3, last_name=Bonham)>]>
```

Because `QuerySet` objects are lazy, you can chain them together. Rerun the previous query, store the result, and then call `.first()` on the result:

```
>>> result = Musician.objects.filter(first_name="John")
>>> result.first()
<Musician: Musician(id=2, last_name=Lennon)>
```

The `.filter()` method is extremely powerful. You can perform matches on substrings or do comparative matching on numeric or date values. All of this is performed by altering the arguments to the method.

4.4.2 Field look-ups

You called `.filter()` to find an exact match using `first_name="John"`, but what if you want everyone who has a first name that begins with *J*? Querying `.filter(first_name="J")` doesn't work, as that looks for an exact match: a first name containing only the letter *J*.

In Python code, if you had a list of names, you could compare each string using an operator. `"John" >= "J"` is `True`, so *John* starts with something grater than or equal to *J*. The problem is you aren't doing this in Python; you're trying to get a database to do it for you.

If you're familiar with SQL, you're probably mumbling "`WHERE` clause" under your breath right now. If you're not familiar with SQL, a `WHERE` clause is a restriction on the bounds of a query—for example, a first name beginning with the letter *J*.

Django supports these kinds of restrictions through *field look-ups*. A field look-up is a modification to a query argument expressed through a double underscore (`__`). For example, the `__startswith` field look-up changes a query from looking for an exact match to looking for fields that begin with the given value. You use a field look-up by appending it to the argument name. To find everyone whose first name begins with *J*, you specify `first_name__startswith`:

```
>>> Musician.objects.filter(first_name__startswith="J")
<QuerySet [<Musician: Musician(id=2, last_name=Lennon)>, <Musician:
➥ Musician(id=3, last_name=Bonham)>]>
```